

```

1  # Note 11: Naive Bayes Algorithm
2  # =====
3
4  # Import Libraries
5  import pandas as pd
6  from sklearn.model_selection import train_test_split
7  from sklearn.naive_bayes import GaussianNB
8  from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
9  # from sklearn.naive_bayes import MultinomialNB
10 # from sklearn.naive_bayes import BernoulliNB
11
12 # Converts text into numerical feature vectors by counting word occurrences
13 # Mainly used to convert text data into numbers for NLP / ML models.
14 from sklearn.feature_extraction.text import CountVectorizer
15
16 # =====
17 # 01. Gaussian Naive Bayes
18 # =====
19
20 # Load dataset
21 df = pd.read_csv("iris.csv")
22
23 # Separate features and target
24 X = df.drop(columns=["Species", "Id"])
25 y = df["Species"]
26
27 # Split data into training and testing sets
28 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
42, stratify=y)
29
30 # Create Gaussian Naive Bayes model
31 model = GaussianNB()
32
33 # Train the model
34 model.fit(X_train, y_train)
35
36 # Make predictions
37 y_pred = model.predict(X_test)
38
39 # Calculate accuracy
40 accuracy = accuracy_score(y_test, y_pred)
41
42 # Print Header
43 print("\n=====")
44 print("01. Gaussian Naive Bayes")
45 print("=====\n")
46 print("01. Accuracy : ", round((accuracy * 100), 2))
47
48 # Confusion Matrix
49 print("\n=====")
50 print("Confusion Matrix")
51 print("=====\n")
52
53 # Print Result
54 print(confusion_matrix(y_test, y_pred))
55
56 # Classification Report
57 print("\n=====")
58 print("Classification Report")
59 print("=====\n")
60 print(classification_report(y_test, y_pred))
61
62 # Predict a new sample
63 new_data = pd.DataFrame([[25, 50000, 170, 1000]], columns=["SepalLengthCm",
"SepalWidthCm", "PetalLengthCm", "PetalWidthCm"])
64 prediction = model.predict(new_data)
65 print("\nPrediction:", prediction)
66
67 # =====
68 # 02. Multinomial Naive Bayes
69 # =====
70

```

```

71 # Load dataset
72 # df = pd.read_csv("data.csv")
73
74 # Example columns:
75 # text -> message/email
76 # target -> spam/not spam
77
78 # Separate input and target
79 # X = df["text"]
80 # y = df["target"]
81
82 # Split data
83 # X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
84 # random_state=42, stratify=y)
85
86 # Convert text into word-count features
87 # vectorizer = CountVectorizer()
88 # X_train = vectorizer.fit_transform(X_train)
89 # X_test = vectorizer.transform(X_test)
90
91 # Create Multinomial Naive Bayes model
92 # model = MultinomialNB()
93
94 # Train model
95 # model.fit(X_train, y_train)
96
97 # Make predictions
98 # y_pred = model.predict(X_test)
99
100 # Accuracy
101 # accuracy = accuracy_score(y_test, y_pred)
102 # print("Accuracy:", accuracy)
103
104 # Confusion Matrix
105 # print("\nConfusion Matrix:")
106 # print(confusion_matrix(y_test, y_pred))
107
108 # Classification Report
109 # print("\nClassification Report:")
110 # print(classification_report(y_test, y_pred))
111
112 # Predict new text
113 # new_text = ["Congratulations! You won a free prize"]
114 # new_text = vectorizer.transform(new_text)
115 # prediction = model.predict(new_text)
116 # print("\nPrediction:", prediction)
117
118 # =====
119 # 03. Bernoulli Naive Bayes
120 # =====
121
122 # Load dataset
123 # df = pd.read_csv("data.csv")
124
125 # Example:
126 # free, offer, win -> binary features
127 # target -> class
128
129 # Separate features and target
130 # X = df.drop("target", axis=1)
131 # y = df["target"]
132
133 # Split data
134 # X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
135 # random_state=42, stratify=y)
136
137 # Create Bernoulli Naive Bayes model
138 # model = BernoulliNB()
139
140 # Train model
141 # model.fit(X_train, y_train)

```

```
141 # Make predictions
142 # y_pred = model.predict(X_test)
143
144 # Accuracy
145 # accuracy = accuracy_score(y_test, y_pred)
146 # print("Accuracy:", accuracy)
147
148 # Confusion Matrix
149 # print("\nConfusion Matrix:")
150 # print(confusion_matrix(y_test, y_pred))
151
152 # Classification Report
153 # print("\nClassification Report:")
154 # print(classification_report(y_test, y_pred))
155
156 # Predict a new sample
157 # new_data = [[1, 1, 0]]
158 # prediction = model.predict(new_data)
159 # print("\nPrediction:", prediction)
```